

CODE SPORT



In this month's column, we discuss a few coding questions.

In last month's column, we had discussed some of the recent state-of-art papers in Natural Language Processing from the Association of Computational Linguistics (ACL) conference. A few readers have asked me for pointers to follow up on recent research papers on different NLP topics. A good resource for this is the website <https://paperswithcode.com/>, which covers not just NLP but the whole of machine learning. For different NLP tasks, you can also browse through papers/techniques that achieve current state-of-art results on various benchmarks.

In this month's column, we take a break from our NLP discussion and instead discuss a few coding questions as practice problems for our readers. When you look at solving any coding problem, think about multiple ways of solving it. And for each of the approaches, think about the time and space complexity of the solution, and how you can optimise it. Remember that the solution's time and space complexity is often dictated by: (a) underlying data structure, and (b) the core underlying algorithm you are using. For instance, if the core underlying algorithm involves a linear scan/search through an array, you are going to end up with at least $O(n)$ time complexity, where 'n' is the size of the array. On the other hand, if your basic operation is a find-minimum on a binary heap, your complexity can reduce to $O(\log N)$. Even with a heap, if you need to search through every element, you will end up with a linear time complexity. So be careful about: (a) the data structures

you choose, and (b) the operations on the data structure you would be invoking. Let us look at an example problem to illustrate this.

You are given an array of unsorted distinct integers. You are asked to find the Kth largest element in the array. What is the time and space complexity of your solution?

The simplest approach is to sort the array in descending order. This incurs a time complexity of $O(\log N)$. And we can simply enumerate the element in the (K-1)st position in the sorted array as the Kth largest element (assuming array elements are indexed from zero). We don't need any additional space, since we can sort the array in place using a standard sorting technique. However, we are doing more work than what has been asked for in the problem, since now we can get all the elements in the right sorted position, not just the Kth largest. So how can we do less work to find just the Kth largest, which will help reduce the time complexity?

This, in turn, suggests that we need to look at techniques other than sorting. We know that we can get the maximum element from a max binary heap in $O(\log N)$. If we keep retrieving the maximum element K times, we get the Kth largest element in the Kth try. Given that it takes $O(N)$ to build a heap, and each extract-max takes $\log(N)$, our time complexity will be $O(k \log N)$. We don't need extra space since we can build the heap in place in the array itself. In this case, we retrieve the first largest element, the second largest element..., all the way up to the Kth largest element. But we are doing extra work again, because we were

